

Homework 2

Student ID: 706709610

Problem 1. *Finding either a Topological Ordering or a Cycle*

Solution:

```
TopoOrCycle (G = {V,E})
  initialize an empty queue Q
  initialize empty in_degree array and set all entries to 0
  initialize an in adjacency list A for n nodes
  initialize empty list T

  for each edge (u,v) in G
    in_degree[v] += 1
    add u to A[v]

  for each node v in G
    if (in_degree[v] == 0)
      push v onto Q

  initialize i to 0
  while (Q not empty)
    pop w from Q and add to T
    for each neighbor u of w
      in_degree[u] -= 1
      if (in degree of u is 0)
        push u onto Q

  if (length of T != n)
    initialize empty visited list
    let x be some node in G with deg[x] != 0
    add x to V
    let w be some node in A[x] with deg[w] != 0
    while (w not in visited)
      add w to visited
      let w be a node in A[w] with deg[w] != 0
    end
    return the reverse list of visited[w:end]
  else
    return T
end
```

Complexity Analysis: The set up is $O(n)$. The first for loop is $O(m)$. The second for loop is $O(n)$. Note that each node can get into Q at most once, so the while loop runs for at most n iterations. For some fixed vertex v the inner for loop runs for each outgoing edge, i.e. out degree of v number of times. Note that each edge (u, v) in G contributes 1 to the out degree of u . So if we sum up the out degrees of all nodes we get exactly m . So the while loop is $O(m)$. If G does have a cycle we would go into the if statement. Finding a node that still has in degree not 0 would be $O(n)$. Note that our while loop will run at most n times. For some node x , getting a node in $A[x]$ which has degree not 0 would need at most in degree of x number of iterations. Following the same logic as done before this comes out to m . Hence our while loop is $O(m)$. To return the backwards list, this is $O(n)$. It follows that the algorithm is $O(n + m)$ $\square$

Problem 2. *Deleting a Node to Delete all Paths*

Solution:

```

DelPath( $G = \{V, E\}$ ,  $s$ ,  $t$ )
  initialize queue  $Q$ , push  $s$  onto  $Q$ 
  initialize distance array of size  $n$  and set all entries to  $\text{inf}$ 
  initialize layer nodes array  $L$ 
  distance[ $s$ ] = 0

  while ( $Q$  is non empty)
    pop  $v$  off  $Q$ 
    for each neighbor  $u$  of  $v$ 
      if (distance[ $u$ ] =  $\text{inf}$ )
        push  $u$  onto  $Q$ 
        distance[ $u$ ] = distance[ $v$ ] + 1
        append  $u$  onto  $L[\text{distance}[u]]$ 
    end
  end

  for each layer  $L_i$  with  $0 < i < \text{distance}[t]$ 
    if  $L_i$  only has one node
      return  $v$ 
    end
  end
end

```

Proof. We now prove that this algorithm correctly finds a node v such that removing v will delete all paths from $s \rightarrow t$.

Claim: A node v exists between s and t such that BFS returns some layer with just v , i.e., there exists

$L_i = \{v\}$. Let $k = \text{distance}(s, t)$. By assumption we have that $k > n/2$. Then since each layer of BFS represents the nodes' distances from s , then we have $L_1, L_2, \dots, L_{k-1}$ are the layers between s and t . Suppose for sake of contradiction that every layer between s and t has at least 2 nodes. We have $k - 1$ layers, which means that across all layers there are at least $2(k - 1) = 2k - 2$ nodes. Note that $k > \frac{n}{2}$ implies that $2k - 2 > n - 2$. This is a contradiction since if we add back s and t nodes, then we will have greater than n nodes across the layers, but we only have n nodes total.

Claim: Removing such a node v also removes all paths from $s \rightarrow t$. As shown above v is in its own layer L_i for some $0 < i < k$. As shown in lecture, every node in L_i is distance i from s . Now taking any path from $s \rightarrow t$ we must have to get to distance i nodes before getting to distance k nodes, i.e., distance can not jump some integer. So since v is the only distance i node, and we are deleting it, it must be the case that all paths no longer work since there is no distance i node to go through.

Claim: The algorithm above correctly finds a v node. The whole first part of the algorithm is just BFS. The last for loop just finds the layer with only one node in it and returns that node. This node is shown to exist by our first claim, so our algorithm works. $\square$

Complexity Analysis: As shown in class BFS has time complexity $O(m + n)$. To see the time complexity of our entire algorithm we just need the last for loop time complexity. Notice that the for loop goes through each layer and checks the size. In the worst case we would check every layer until the last one, this amounts to checking every node in the graph. Since there are n nodes, then the for loop is $O(n)$. It follows that the entire algorithm is of $O(n + m)$. $\square$

Problem 3. *Butterfly Classification*

Solution:

```

Butterfly (L[1,same], L[1,diff], ..., L[n,same], L[n,diff])
    initialize n sized label array and initialize each entry to 0
    initialize empty queue Q
    pick node 1, label[1] = A, push 1 onto Q

    while (Q is not empty)
        pop v from Q
        for each node u in L[v,same]
            if (label[u] == 0)
                push u onto Q
                label[u] = label[v]
            else if (label[u] != label[v])
                return "inconsistent"

```

```

    end

    for each node u in L[v,diff]
        if (label[u] == 0)
            push u onto Q
            label[u] = opposite of label[v]
        else if (label[u] == label[v])
            return "inconsistent"
        end
    end
end
return "consistent"
end

```

We now prove that the algorithm correctly determines whether the judgements are consistent or inconsistent.

Proof. We wish to show that the Butterfly($L_{1,\text{same}}, L_{1,\text{diff}}, \dots, L_{n,\text{same}}, L_{n,\text{diff}}$) outputs consistent if and only if the judgements are consistent.

$\Rightarrow$. Suppose the algorithm outputs consistent. We wish to show the judgements are consistent, i.e. the nodes which are marked as same have the same labels and the ones marked as different have opposite labels. Notice that the algorithm will return “inconsistent” only if the existing label violates one of its edges. Also notice that we are using BFS, so every node will be checked and also its edges since BFS checks all neighbors. Since we never return “inconsistent”, no node’s label violates any of its edges. Explicitly we have that for any $v \in V$: (1) for any $u \in L_{v,\text{same}}$, the label of v is the same as the label of u and (2) for any $u \in L_{v,\text{diff}}$, the label of v is opposite of the label of u . This is what we wanted to prove.

$\Leftarrow$. Suppose the judgements are consistent, i.e., there exists a valid labeling L . We wish to show the algorithm outputs “consistent.” To show this, we will show that each node’s label matches with L , possibly after swapping groups A and B . We proceed by induction and suppose WLOG that node 1 is in group A . Our algorithm first assigns node 1 to group A in the set up phase. Now suppose that our labeling is consistent with L up to its k -th assignment v . Then when we pop v from Q and check its neighbor u , one of two things happen: (1) There is no label on u , in which case the algorithm will correctly assign a correct label consistent with L and (2) There is a label on u which means that it was already assigned at one point and hence by our inductive hypothesis it is correct. $\square$

Complexity Analysis: The algorithm is essentially BFS with some modifications. So we expect $O(m + n)$. The main difference is that we split the for loop into two. But something to notice is that the combined iterations of the two for loops will be the same amount as in regular BFS. This is because in BFS we check all neighbors, but here we just split the neighbors into two disjoint groups.

The rest of the operations are $O(1)$, so it follows that our algorithm has the same time complexity of BFS, i.e. $O(m + n)$. $\square$

Problem 4. *Number of Shortest Paths.*

Solution:

```

NumShortestPaths( $G=\{V,E\}$ ,  $v$ ,  $w$ )
    initialize  $n$ -sized array  $P$  with all entries 0
    initialize  $n$ -sized distance array  $D$  with all entries inf
    initialize queue  $Q$ , push  $v$  onto  $Q$ 
     $P[v] = 1$ 
     $D[v] = 0$ 

    while( $Q$  is not empty)
        pop  $x$  from  $Q$ 
        if ( $x == w$ )
            return  $P[w]$ 
        for each neighbor  $u$  of  $x$ 
            if ( $D[u] == \text{inf}$  or  $D[u] == (D[x] + 1)$ )
                 $P[u] += P[x]$ 
                if ( $D[u] == \text{inf}$ )
                     $D[u] = D[x] + 1$ 
                    push  $u$  onto  $Q$ 
        end
    end
end

```

Proof. We claim that for every node u with $D[u] = i$, once u is popped from Q , $P[u]$ holds the correct number of shortest paths. We proceed by induction on i . Base case is $i = 0$. This is v , which is assigned to have $P[v] = 1$ which is definitively correct. Now suppose that all nodes with distance $i - 1$ have their path number correct. Take any distance i node u . Then u neighbors a family of distance $i - 1$ nodes. Notice that all such neighbors x of u are in a different layer then they will be popped before u is popped from the queue. The algorithm acumulates all such neighboring nodes path number $P[x]$, which are correct by our inductive hypothesis. This summation defines what the path number is for a certain node, so $P[u]$ will correctly have the path number. Once all the neighboring nodes are popped, we will have that there is nothing left to sum. So once u is popped it will have the correct path number. $\square$

Complexity Analysis: Notice that we are essentially using BFS with some modifications. But all of these modifications happen in $O(1)$ time complexity. It follows that our algorithm has the same time complexity of BFS, i.e. $O(m + n)$. $\square$

Problem 5. *Births and Deaths*

Solution:

```

BirthDeath(People P_1, P_2, ..., P_n, Facts F)
  create 2n nodes b_1, b_2, ..., b_n, d_1, ..., d_n for graph G
  initialize empty edge set for graph G

  for i in {1, ..., n}
    add directed edge (b_i, d_i)
  end

  for each fact f in F
    if (f is "P_i died before P_j was born")
      add directed edge (d_i, b_j)
    if (f is "P_i died before P_j died")
      add directed edge (d_i, d_j)
    if (f is "P_i overlaps P_j")
      add two directed edges if they don't exist already (b_i, d_j) and (b_j, d_i)
    end
  end

  result = TopoOrCycle(G)

  if (result is a cycle)
    return "inconsistent"

  for node in result
    date(node) = position in topological order of result
  end
end

```

Proof. We wish to show that the facts are consistent if and only if the algorithm returns a valid date assignment. For understanding purposes, we should state the definitions of edges in the graph G :

- $b_i \rightarrow d_i$ is for every i since birth comes before death.
- $d_i \rightarrow b_j$ is fact 1: person i dies before person j is born.
- $d_i \rightarrow d_j$ is fact 2: person i dies before person j dies.
- $d_i \rightarrow b_j$ and $b_i \rightarrow d_j$ is fact 3: person i and j lives overlap (proven below)

Claim: if P_i and P_j overlap then we must have directed edges (b_j, d_i) and (b_i, d_j) . Notice that if P_i and P_j don't overlap then we have two situations $d_i \leq b_j$ or $d_j \leq b_i$. So if they do overlap it should be the negation of that. We have, $b_j < d_i$ and $b_i < d_j$. This corresponds to directed edges (b_j, d_i) and (b_i, d_j)

$\Leftarrow$. Suppose that the algorithm returns a valid date assignment. This means that G has no cycle. Since **TopoCycle** is correct (we proved it in problem 1), then it returns a valid topological ordering of G . This means that for every edge $x \rightarrow y$ we have that $\text{date}(x) < \text{date}(y)$. We will go through each fact type:

- A $b_i \rightarrow d_i$ means that person i was born before they died. It follows that $\text{date}(b_i) < \text{date}(d_i)$.
- A $d_i \rightarrow b_j$ means that person i died before person j was born. So we get $\text{date}(d_i) < \text{date}(b_j)$ which is consistent.
- A $d_i \rightarrow d_j$ means that person i died before person j died. So we get $\text{date}(d_i) < \text{date}(d_j)$ which is consistent.
- A $b_j \rightarrow d_i$ and $b_i \rightarrow d_j$ means that person i overlaps with person j and $\text{date}(b_j) < \text{date}(d_i)$ and $\text{date}(b_i) < \text{date}(d_j)$ which makes sense.

Since all facts are consistent with our date function and our date function is valid, then it follows that all facts are consistent.

$\Rightarrow$. Suppose the facts are consistent. So there exists a valid date for the death and birth of each person, call it $d^*(b_i)$ and $d^*(d_i)$. Suppose G is inconsistent, then G contains a cycle:

$(x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_k \rightarrow x_1)$. But since the date function d^* is valid we must have that $d^*(x_1) < d^*(x_2) < \dots < d^*(x_k) < d^*(x_1) \Rightarrow d^*(x_1) < d^*(x_1)$. Contradiction.

□

□